



UNSW
SYDNEY

COMP4141

Theory of Computation

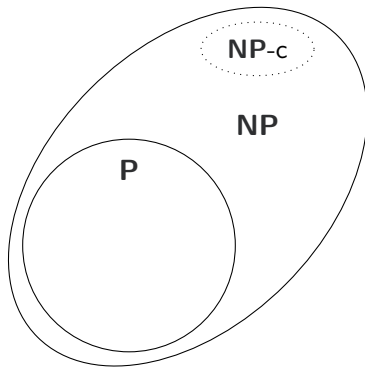
Lecture 15: Complexity Classes

Recap

NP-complete:

- SAT, 3SAT, 4SAT
- VERTEX COVER
- CLIQUE
- HAMILTONIAN PATH, HAMILTONIAN CYCLE
- TRAVELLING SALESMAN
- SUBSET SUM
- INTEGER LINEAR PROGRAMMING, 0-1 ILP
- 3COL

Complexity classes



Outline

coNP

EXPTIME

Space complexity

PSPACE

Logarithmic space: **L** and **NL**

These and many more can be found at: [Complexity Zoo](#)

coNP

P is closed under complementation

It is not known whether **NP** is closed under complementation.

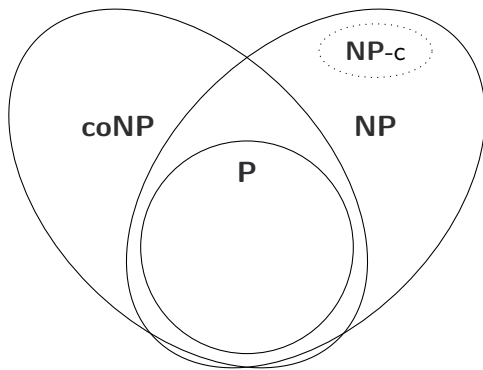
Definition

coNP is the set of all languages $\Sigma^* \setminus A$, where A is a language in **NP**.

Trivially:

$$\mathbf{P} \subseteq (\mathbf{coNP} \cap \mathbf{NP})$$

Complexity classes



coNP example

VALIDITY

Input: A boolean formula φ

Question: Is φ true for all valuations?

Theorem

VALIDITY *is* coNP-complete

Proof: Exercise.

coNP Characterizations: Universal acceptance

Recall $L^\forall(A)$ for NFAs.

Say that the computation tree of a nondeterministic machine M *universally accepts input* w if every branch of the computation tree of M is accepting.

Let $L^\forall(M)$ be the set of w such that M universally accepts w .

Theorem

coNP is the set of all languages $L^\forall(M)$ where M is nondeterministic polynomial time Turing machine.

coNP Characterizations: Refuters

A **refuter** for a language A is an algorithm V , where

$$\overline{A} = \{ w \mid V \text{ rejects } \langle w, c \rangle \text{ for some string } c \}$$

We measure the running time of a refuter only in terms of the length of w , not that of the **certificate** (or **proof**) c .

Theorem

coNP is the class of languages that have polynomial time refuters.

Outline

coNP

EXPTIME

Space complexity

PSPACE

Logarithmic space: **L** and **NL**

These and many more can be found at: [Complexity Zoo](#)

Exponential time

The **P** vs **NP** problem focuses on the question

*“Are there ‘efficient’ algorithms for **NP** problems?”*

What alternatives are there if the answer is no?

“Exponential time” is intuitively a natural candidate.

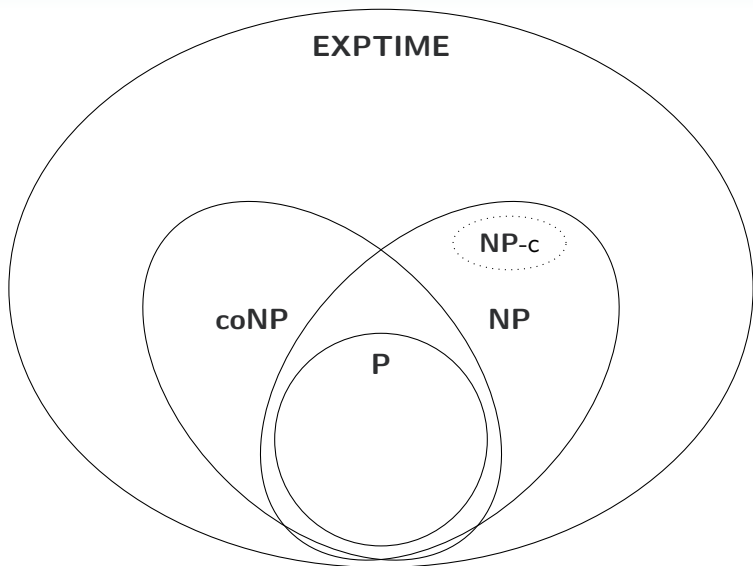
Definition

$$\text{EXPTIME} = \bigcup_{k \in \mathbb{N}} \text{TIME}(2^{n^k})$$

NB

EXPTIME $\neq$ **TIME**(2^n): *the latter is not closed under polynomial time reductions.*

Complexity classes



EXPTIME example

HALT

Input: A Turing Machine M

Question: Does M halt?

EXPTIME example

BOUNDED HALT

Input: A Turing Machine M and an integer k

Question: Does M halt in at most k steps?

Theorem

BOUNDED HALT *is* **EXPTIME**-complete

Proof: Exercise.

$P \neq \text{EXPTIME}$

An immediate consequence of the Time Hierarchy Theorem:

Theorem

$$P \subsetneq \text{EXPTIME}$$

NP = EXPTIME?

Argument against:

Many “natural” complexity classes between **P** and **EXPTIME**:

- Sub-exponential time: **TIME**($2^{o(n)}$) (e.g. **TIME**($2^{\sqrt{n}}$))
- Quasi-polynomial time: **TIME**($n^{\omega(1)}$) (e.g. **TIME**($n^{\log n}$))
- **PSPACE**
- Polynomial hierarchy (e.g. **NP** with **NP** oracles)

NP = EXPTIME?

Argument against:

Many “natural” complexity classes between **P** and **EXPTIME**:

- Sub-exponential time: **TIME**($2^{o(n)}$) (e.g. **TIME**($2^{\sqrt{n}}$))
- Quasi-polynomial time: **TIME**($n^{\omega(1)}$) (e.g. **TIME**($n^{\log n}$))
- **PSPACE**
- Polynomial hierarchy (e.g. **NP** with **NP** oracles)

Argument for:

Exponential time hypothesis

SAT cannot be solved in sub-exponential time.

Outline

coNP

EXPTIME

Space complexity

PSPACE

Logarithmic space: **L** and **NL**

These and many more can be found at: [Complexity Zoo](#)

Space Complexity Classes

Recall:

Definition

Let $f : \mathbb{N} \rightarrow \mathbb{N}$. Define the *space complexity classes*, **SPACE**($f(n)$) and **NSPACE**($f(n)$) to be the collection of all languages that are decidable by an $O(f(n))$ space TM, resp., NTM.

$\text{SAT} \in \mathbf{SPACE}(n)$

Remember $\text{SAT} = \{ \langle \varphi \rangle \mid \varphi \text{ is a satisfiable Boolean formula} \}$.

Theorem

$\text{SAT} \in \mathbf{SPACE}(n)$.

Proof: Formatif Task

Example: ALL_{NFA}

Consider the decision problem:

ALL_{NFA}

Input: An NFA A

Question: Is $L(A) = \Sigma^*$?

Currently, it's known neither whether $ALL_{NFA} \in \mathbf{NP}$ nor whether $ALL_{NFA} \in \mathbf{coNP}$.

ALL_{NFA} $\in$ NSPACE(n)

On input $\langle A \rangle$, where $A = (Q, \Sigma, \delta, q_0, F)$ is an NFA:

- 1 Place a marker on q_0
- 2 Repeat $2^{|Q|}$ times:
 - Let $m \subseteq Q$ be the positions of markers.
 - (nondeterministically) pick an $a \in \Sigma$
 - change m to $\bigcup_{q \in m} \delta(q, a)$.
 - If $m \cap F = \emptyset$, *ACCEPT*.
- 3 *REJECT*

Note that M may use exponential time but linear space only.

$2^{|Q|}$ iterations are needed to ensure we can reach all possible patterns of markers on states.

Why didn't we introduce **coNSPACE**?

SPACE vs. NSPACE

Theorem (Savitch)

For any function $f : \mathbb{N} \rightarrow \mathbb{N}$, where $f(n) \geq n$,

$$\text{NSPACE}(f(n)) \subseteq \text{SPACE}((f(n))^2) .$$

To prove this, we need to simulate an $f(n)$ space NTM deterministically.

Bad proof idea: try all branches of the NTM's computation, one after the other. This requires storing a stack of choices made along the branch. A branch that uses $f(n)$ space may run for $2^{O(f(n))}$ steps, and each step may require recording a choice. So this approach may use $2^{O(f(n))}$ space.

Savitch's Proof Idea

Use binary (recursive) search to find an accepting branch of the NTM's computation.

After modifying the NTM to one that clears its tape before accepting, also the final (accepting) configuration $q_{\text{accept}} \in$ is unique.

The initial configuration for input w is $q_0 w$.

The number of different configurations of the NTM

$N = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ in $f(n)$ space is at most:

$$k(n) = \underbrace{|\Gamma|^{f(n)}}_{\text{tape content}} \cdot \underbrace{(f(n) + 2)}_{\text{head position}} \cdot \underbrace{|Q|}_{\text{state}} = 2^{O(f(n))}$$

Savitch's Proof Idea cont.

We build a TM M that decides $L(N)$ in **SPACE** $((f(n))^2)$.

On input w it outputs the result of

$$\text{CANYIELD}(q_0 w, q_{\text{accept}}, k(|w|))$$

where $\text{CANYIELD}(c_1, c_2, t)$ is a deterministic recursive procedure that decides whether N can yield configuration c_2 from c_1 within t steps.

Savitch's Proof Idea cont.

CANYIELD(c_1, c_2, t)

- If $t = 1$:
 - If $c_1 = c_2$ or if c_1 can yield c_2 in N , *ACCEPT*.
 - else *REJECT*.
- For each of the $k(|w|)$ configurations c_m of N on input w using space $f(|w|)$:
 - Run CANYIELD($c_1, c_m, \lfloor \frac{t}{2} \rfloor$).
 - Run CANYIELD($c_m, c_2, \lceil \frac{t}{2} \rceil$).
 - If both recursive calls accepted, *ACCEPT*.
- *REJECT*.

Savitch's Proof Idea cont.

How much space does M use?

Because each recursive call of $\text{CANYIELD}(c_1, c_2, t)$ roughly halves the value of the third argument the maximum recursion depth is $\lceil \log_2 t \rceil$. The initial value for t is $k(|w|) = 2^{O(f(|w|))}$.

So the maximum recursion depth is $O(f(|w|))$.

Each recursive call should store the values of the arguments on a stack. The size of such a stack frame is $O(f(|w|))$.

Altogether M thus uses $O((f(|w|)^2))$ space.

NB

One technical problem is that we don't know whether it is safe to assume that $f(n)$ can be computed in the space given. To avoid this problem we could modify M to try $f(n) = 1, 2, \dots$. We'd modify $\text{CANYIELD}(c_1, c_2, t)$ to stop and complain if the current configuration size is exceeded and then start over with the next size.

Outline

coNP

EXPTIME

Space complexity

PSPACE

Logarithmic space: **L** and **NL**

These and many more can be found at: [Complexity Zoo](#)

PSPACE

Definition

$$\mathbf{PSPACE} = \bigcup_{k \in \mathbb{N}} \mathbf{SPACE}(n^k)$$

We could also define $\mathbf{NPSPACE} = \bigcup_{k \in \mathbb{N}} \mathbf{NPSPACE}(n^k)$, however

$$\mathbf{PSPACE} = \mathbf{NPSPACE}$$

by Savitch's theorem.

Relationships

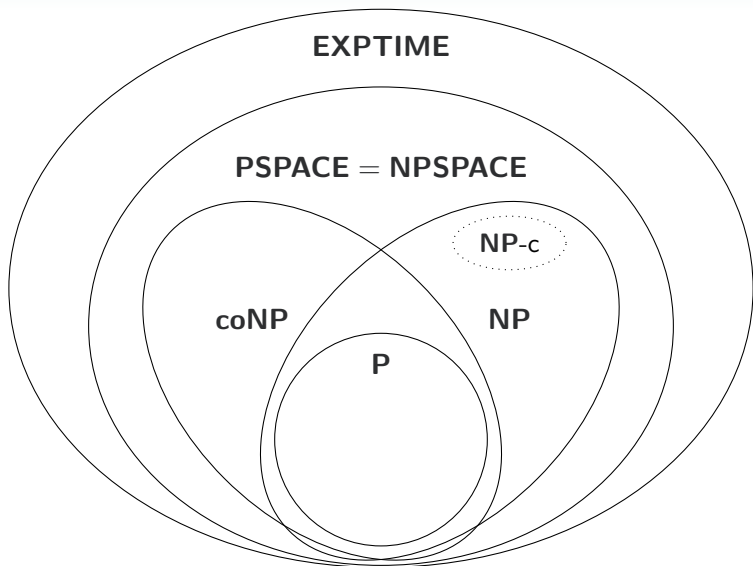
$$\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{PSPACE} = \mathbf{NPSPACE} \subseteq \mathbf{EXPTIME}$$

where $\mathbf{EXPTIME} = \bigcup_{k \in \mathbb{N}} \mathbf{TIME}(2^{n^k})$.

Proof of $\mathbf{PSPACE} \subseteq \mathbf{EXPTIME}$.

A TM that uses $f(n)$ space and halts will pass through no more than $f(n)2^{O(f(n))}$ configurations, all of which must be distinct. $\square$

Complexity classes



PSPACE-Completeness

Definition

A language B is **PSPACE-complete** if

- 1 $B \in \mathbf{PSPACE}$
- 2 B is **PSPACE-hard**, i.e., every $A \in \mathbf{PSPACE}$ is polynomial time reducible to B .

Quantified Boolean Formulas

We can extend propositional logic over atomic propositions *Prop* by adding quantifiers:

$$\begin{aligned} \varphi \rightarrow p \mid \varphi \wedge \varphi \mid \neg \varphi \mid (\varphi) \mid \forall p(\varphi) \mid \exists p(\varphi) \\ p \rightarrow x \mid y \mid \dots \end{aligned}$$

The semantics for the extension can be given by treating quantifiers as abbreviations:

$$\begin{aligned} \forall x(\varphi) &= \varphi[\text{TRUE}/x] \wedge \varphi[\text{FALSE}/x] \\ \exists x(\varphi) &= \varphi[\text{TRUE}/x] \vee \varphi[\text{FALSE}/x] \end{aligned}$$

where $\varphi[t/x]$ is the result of substituting t for all (free) occurrences of x in φ .

QBF is PSPACE-complete

A *sentence* of quantified boolean logic is a formula that

- 1 has no free (unquantified) variables, and
- 2 is in prenex normal form (i.e., all quantifiers are at the front).

The truth value of a sentence does not depend on an assignment π .

Theorem

$\text{QBF} = \{ \langle \varphi \rangle \mid \varphi \text{ is a true sentence} \}$ is **PSPACE-complete**.

Proof: $\text{QBF} \in \text{PSPACE}$

$T =$ “On input $\langle \varphi \rangle$, where φ is a sentence:

- 1 If φ is quantifier-free, evaluate it and *accept* if it's true and *reject* otherwise.
- 2 If φ is $\exists x (\psi)$, recursively call T on $\psi[\text{FALSE}/x]$ and $\psi[\text{TRUE}/x]$. *Accept*, if either result is *accept* and *reject* otherwise.
- 3 If φ is $\forall x (\psi)$, recursively call T on $\psi[\text{FALSE}/x]$ and $\psi[\text{TRUE}/x]$. *Accept*, if both results are *accept* and *reject* otherwise.”

Proof: QBF is **PSPACE**-hard

To prove hardness we recycle two proof ideas: Cook's to encode TM configurations and Savitch's to compress search trees.

Let $A \in \mathbf{PSPACE}$, say, $A \in \mathbf{SPACE}(n^k)$ as witnessed by TM M .

On input w we build a sentence

$$\varphi_{C_{\text{start}}, C_{\text{accept}}, h}$$

that is true iff M accepts w in at most $h = 2^{dn^k}$ steps where M has no more than h possible configurations of length n^k when computing on a word of length $n = |w|$.

The sentence $\varphi_{c_1, c_2, t}$

encodes ingredients of a TM configuration of length up to n^k as in the Cook-Levin theorem using $\ell = O(n^k)$ variables.

For $t = 1$ we encode that $c_1 = c_2$ or c_1 yields c_2 (in a single step).

For $t > 1$ we try

$$\varphi_{c_1, c_2, t} = \exists \mathbf{m} \varphi_{c_1, \mathbf{m}, t/2} \wedge \varphi_{\mathbf{m}, c_2, t/2}$$

where $\exists \mathbf{m}$ abbreviates $\exists m_1, \dots, \exists m_\ell$ —we quantify over a whole row of the tableau used to encode the computation of M .

This almost works; the only flaw is that the formula's size is exponential in n

The sentence $\varphi_{c_1, c_2, t}$ cont.

To fix this, we replace the conjunction by a quantification:

$$\varphi_{c_1, c_2, t} = \exists \mathbf{m} \left(\forall (\mathbf{c}_3, \mathbf{c}_4) \in \{(\mathbf{c}_1, \mathbf{m}), (\mathbf{m}, \mathbf{c}_2)\} \left(\varphi_{\mathbf{c}_3, \mathbf{c}_4, t/2} \right) \right)$$

— QED —

Games

Game theory nowadays plays an important role in economics, politics, and computer science.

Let's consider two-player turn-based games such as Tic-Tac-Toe, Checkers, Chess, Go, Geography Game, etc. only.

A *position* is a comprehensive description of the game state.

Example

×		
	○	
○		×

is such a description for Tic-Tac-Toe (assuming that the player placing × always moves first).

Histories of moves are always usable as positions, however, they may be not concise enough for practical purposes.

Graphs and Game Trees

In general, the possible positions of (finite) games form directed graphs where the edges are sometimes labelled with the player who can move from the position at the source of the edge to the position at the destination.

When we use histories as positions then graphs become trees. In two-player games where players alternate, odd level tree moves belong to one player whereas even level moves belong to the other.

The leaves in such a tree describe final positions of the game. These leaves are marked depending on who wins the game when it ends in this position.

Strategies

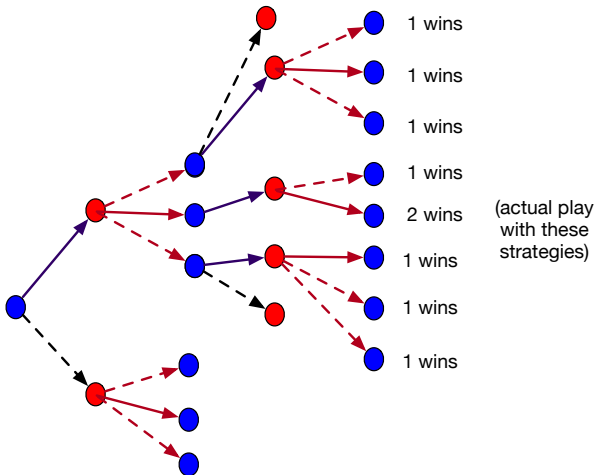
A strategy for player i is a mapping from nodes of the game tree (i.e. histories of the play so far) at which the player takes has a turn to move, to a valid move for the player.

A *winning strategy* for player i from a position is a strategy that guarantees that for *all* strategies of the opponent, the resulting play of the game ends with player i winning.

A winning strategy selects *one* move at each position, but it needs to work for *all* strategies of the opponent.

Turn: ● Player 1 —————→ move selected by player 1 strategy

 ● Player 2 —————→ move selected by player 2 strategy



Connection to Logic

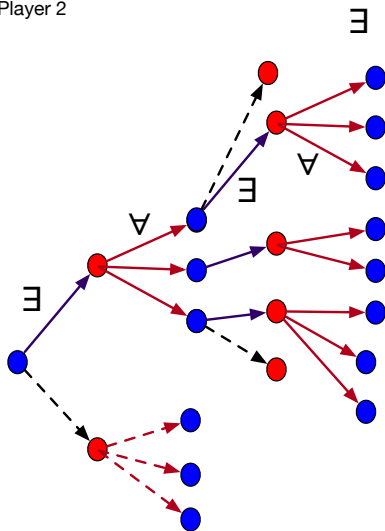
The question “does player E have a winning strategy in position p of game G ” can often be re-phrased as a quantified Boolean formula

$$\exists m_1 (\forall r_1 (\exists m_2 (\dots \varphi)))$$

where the m_i and r_i range over player E 's, resp. A 's moves and φ expresses that E wins.

Turn: ● Player 1 → move selected by player 1 strategy

 ● Player 2



Formula Game

Let's consider a simple formula game. Let ψ be a Boolean formula over propositions $x_1, \dots, x_k$. In move i a player chooses a truth value for x_i . Player E takes the odd turns and player A takes the even turns. Player E wins if the formula turns out true, otherwise A wins.

Player E has a winning strategy iff the quantified Boolean formula

$$\exists x_1 (\forall x_2 (\exists x_r (\dots \psi)))$$

is true.

Formula Game cont.

Since it is pretty much the same as QBF it follows that

$$\{ \langle \psi \rangle \mid E \text{ has a winning strategy for } \psi \}$$

is also **PSPACE**-complete.

Sipser puts the quantifiers into ψ but this doesn't make a difference because we assume an order on the propositions and from that the quantifier sequence follows.

Geography

Sipser writes that children play a game in which one starts with a city name and the next child always has to come up with a *new* city name that starts with the last letter of the previous one. The child who cannot name another city (not previously named) in this sequence loses.

We could re-phrase this as a graph problem by making all the known city names nodes and have edges from a name wa to bv iff $a = b$.

Or we could generalise this even further to an arbitrary directed graph and forget about the city names.

$$GG = \{ \langle G, b \rangle \mid \text{Kid 1 has a w. s. in } G \text{ starting at } b \}$$

Geography cont.

Theorem

GG is **PSPACE**-complete.

Proof of $\text{GG} \in \text{PSPACE}$.

That GG is in **PSPACE** follows from this **PSPACE** algorithm:

On input $\langle (V, E), b \rangle$:

- 1 Reject if b has no successor nodes in (V, E) because Player 1 loses.
- 2 Remove b from (V, E) : let $V_1 = V \setminus \{b\}$ and $E_1 = E \cap (V_1 \times V_1)$.
- 3 For each b_1 such that $(b, b_1) \in E$ run M on $\langle (V_1, E_1), b_1 \rangle$.
- 4 If all of these recursive calls accept, then Player 2 has a winning strategy in the game so reject, otherwise accept.

At each point we only need to store the description of the game (i.e. the input) and the stack of crossed out nodes. □

Geography cont.

Proof of GG is PSPACE-hard.

by reduction from QBF.



Real Games

Real games such as Chess and Go

([https://en.wikipedia.org/wiki/Go_\(game\)](https://en.wikipedia.org/wiki/Go_(game)))

have fixed board sizes and thus finite numbers of positions.

Therefore the **PSPACE** arguments don't really apply.

But we can generalize these games to $n \times n$ boards and get results like:

Theorem

The problem of deciding if a given $n \times n$ GO position is winning is PSPACE-hard.

(GO is PSPACE-hard, Lichtenstein & Sipser, JACM 1980)

Outline

coNP

EXPTIME

Space complexity

PSPACE

Logarithmic space: **L** and **NL**

These and many more can be found at: [Complexity Zoo](#)

Sublinear Space

Consider 2-tape TMs where the tape containing the input tape is read-only.

Change the definition of space complexity to ignore the input tape.
(*Remark:* This matters at most for sublinear space.)

Definition

$$\mathbf{L} = \mathbf{SPACE}(\log n)$$

$$\mathbf{NL} = \mathbf{NSPACE}(\log n)$$

Example

$\{ 0^k 1^k \mid k \in \mathbb{N} \} \in \mathbf{L}$: Use a single binary counter initialised to 0 to first count the 0s up and then the 1s down.

S-T REACHABILITY

Recall that

S-T REACHABILITY

Input: A directed graph G and vertices s, t

Question: Is t reachable from s in G ?

is in **P**.

S-T REACHABILITY

Here is an **NL** algorithm:

On input $\langle (V, E), s, t \rangle$:

- ① Store $v \leftarrow s$ on the 2nd tape
- ② Repeat up to $|V| - 1$ times:
 - non-deterministically guess v' with $(v, v') \in E$
 - if $v' = t$, *ACCEPT* else store $v \leftarrow v'$
- ③ *REJECT*

Whether $\mathbf{L} = \mathbf{NL}$ is open. Whether S-T REACHABILITY $\in \mathbf{L}$ is unclear, but the **undirected** version is known to be in $\mathbf{L}$ (Reingold 2005).

Log Space Reducibility

Polynomial-time reducibility ($\leq_P$) is too coarse a measure to define **NL**-completeness. Instead, we'll use *log space reducibility* ($\leq_L$) based on log space transducers:

Definition

A *log space transducer* is a 3-tape TM with

- 1 a read-only *input* tape,
- 2 a read-write *working* tape, and
- 3 a write-only *output* tape

that uses only $O(\log n)$ space on the working tape.

Theorem

If $A \leq_L B$ and $B \in \mathbf{L}$, then $A \in \mathbf{L}$.

NL-completeness

Definition

A language B is **NL-complete** if $B \in \mathbf{NL}$ and for every $A \in \mathbf{NL}$ we have $A \leq_L B$.

Theorem

S-T REACHABILITY *is* **NL-complete**.

Corollary

$\mathbf{NL} \subseteq \mathbf{P}$

Certificate Definition of **NL**

Recall that **NP** is the class of languages for which **P** verifiers exist. A similar characterisation can be given for **NL**.

Theorem

$L \in \mathbf{NL}$ if L has a logspace verifier, that is, a 3-tape TM M and a polynomial p such that for all x there exists a certificate u of size $p(|x|)$ and M accepts iff $x \in L$ when started as follows:

- 1 tape 1 is read-once (from left to right) and contains the certificate u ,*
- 2 tape 2 is read-only and contains the input x , and*
- 3 tape 3 is read-write work tape of size $O(\log|x|)$.*

NL = Logspace verifiable

Proof.

As for the two characterisations of **NP**, we show one direction by using a description of an accepting run of the NTM as certificate and the other direction by guessing the certificate symbol-by-symbol when we need it. □

None of this is in [Sipser2006].

Logspace verifier example

Example

To show once again that $S\text{-}T \text{ REACHABILITY} \in \mathbf{NL}$ we let the certificate for $\langle ((V, E), s, t) \rangle \in S\text{-}T \text{ REACHABILITY}$ be a list $[v_0, \dots, v_k]$ of nodes forming an acyclic path from s to t in (V, E) . The verifier checks that

- ❶ $v_0 = s$,
- ❷ $(v_j, v_{j+1}) \in E$, for all $0 \leq j < k$, and
- ❸ $v_k = t$.

This takes at most logspace because (a) it suffices to store 2 nodes and (b) node names are binary representations of $1, \dots, |V|$.

Immerman-Szelepcsényi Theorem

Theorem (Immerman-Szelepcsényi)

NL = coNL

Proof.

Show $\overline{\text{S-T REACHABILITY}} \in \mathbf{NL}$ by providing a logspace verifier and certificates.

On input $\langle (V, E), s, t, u \rangle$ our verifier uses two procedures to certify that:

- 1 $v \notin C_i$ given $|C_i|$
- 2 $|C_i| = c$, given $|C_{i-1}|$

where $C_0 = \{s\}$ and $C_{i+1} = C_i \cup E(C_i)$, i.e., C_i is the set of nodes reachable from s in at most i steps.

Applying the second procedure $|V| - 1$ times yields the number of nodes reachable from s and the first procedure can then certify $t \notin C_{|V|-1}$. $\square$

Proof details I

What can we do with a logspace verifier?

Given $v \in V$ **and** $i \leq |V|$ **we can certify** $v \in C_i$.

Same as for S-T REACHABILITY plus counting steps.

Given $v \in V$, $i \leq |V|$, **and** $|C_i|$ **we can certify** $v \notin C_i$.

cert = ordered list of $|C_i|$ certificates for the $u \in C_i$.

Check that:

- 1 each sub-certificate is valid as per the previous point,
- 2 certificates are ordered,
- 3 none of the certified nodes is v , and
- 4 the number of certificates is $|C_i|$

Proof details II

Given $v \in V$, $i \leq |V|$, **and** $|C_{i-1}|$ **we can certify** $v \notin C_i$.

Ordered list of $|C_{i-1}|$ certificates for the $u \in C_{i-1}$.

Check that:

- 1 each sub-certificate is valid as per the previous point,
- 2 certificates are ordered,
- 3 the certified nodes are neither v nor neighbours of v , and
- 4 the number of certificates is $|C_{i-1}|$.

Proof details III

Given $|C_{i-1}|$ **and** c **we can certify** $|C_i| = c$.

cert = ordered list of $|V|$ certificates for the $v \in V$ of either $v \in C_i$ or $v \notin C_i$ as described above.

Check that:

- 1 each sub-certificate is valid as per the previous points while counting the total number of certificates and the number of $v \in C_i$ certificates
- 2 Accept if those counts are $|V|$ and c , respectively.

Complexity classes

